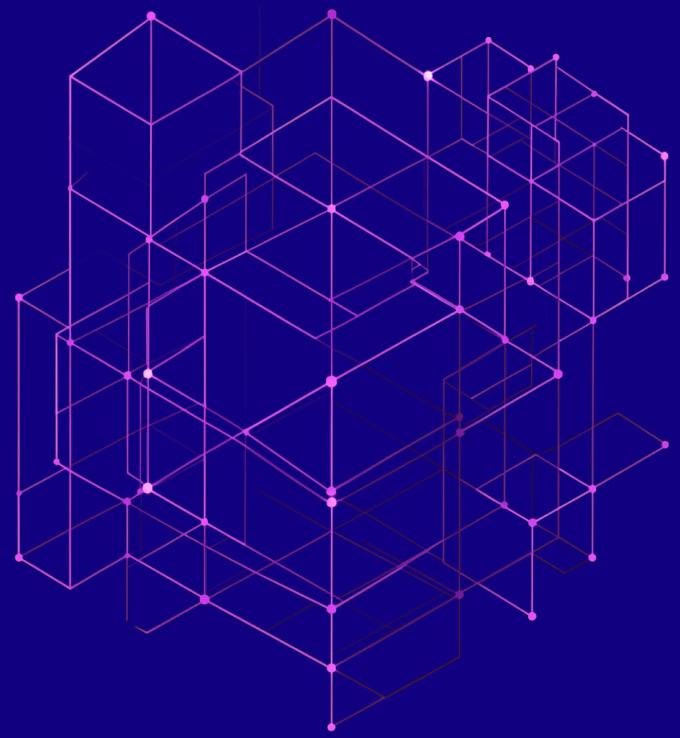
UNIVERSIDAD COOPERATIVA DE COLOMBIA

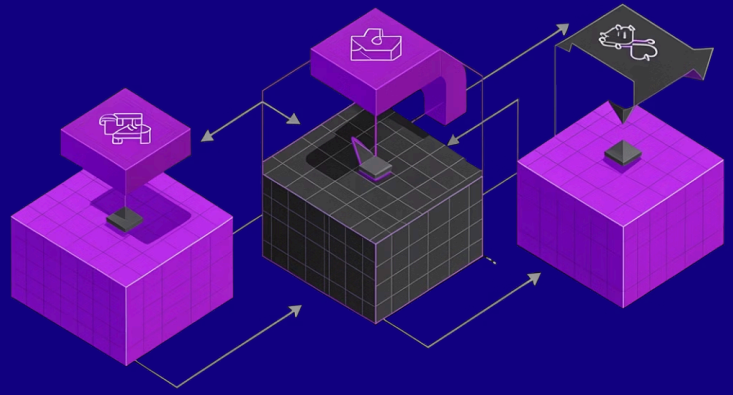
Programación de Entornos Multimedia

Geometrías, Materiales y Transformaciones 3D en Three.js

Mg. Gabriel Guerrero Arellano

Objetivo: Dominar la construcción de mallas 3D (Meshes), la aplicación de materiales/texturas y la manipulación del espacio mediante transformaciones matemáticas.





Hoja de Ruta de la Sesión

Plan de trabajo para las 3 horas de clase

Bloque 1 — Hora 1

Anatomía de la Malla 3D (Mesh) y Geometrías Primitivas.

Bloque 2 — Hora 2

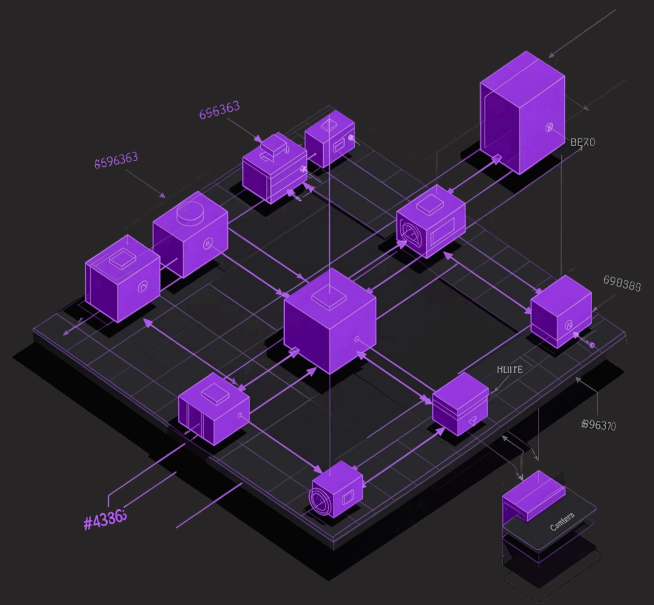
Materiales, Luces y Carga de Texturas (PBR Intro).

Bloque 3 — Hora 3

Transformaciones 3D (Posición/Rotación/Escala) y Laboratorio Interactivo con lil-gui.

Reanimando la Escena Básica (Semana 1 Recap)

Los tres pilares de Three.js antes de crear objetos



Escena (THREE.Scene)

El contenedor virtual 3D donde residen todos los objetos.

Cámara (THREE.PerspectiveCamera)

El punto de vista del usuario (FOV, Aspect Ratio, Near, Far).

Renderizador (THREE.WebGLRenderer)

El motor que dibuja la escena en el canvas HTML5.

Concepto Fundamental: ¿Qué es una Malla (Mesh)?

La unión de forma y apariencia en el espacio tridimensional

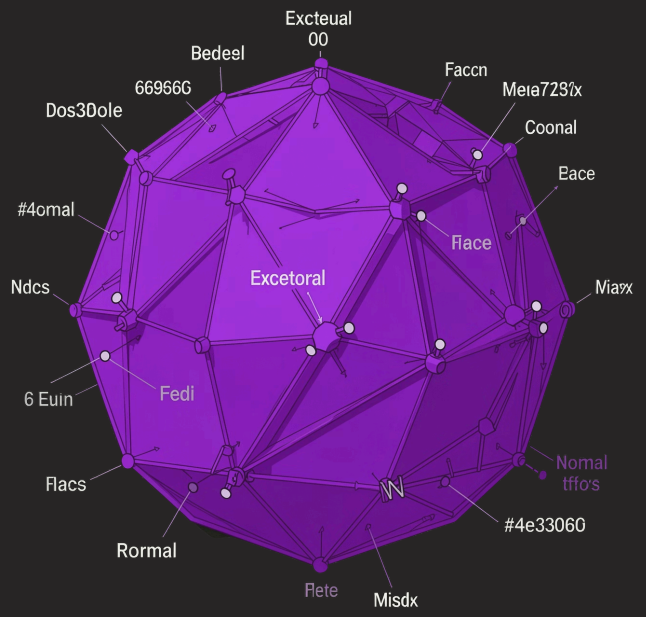
GEOMETRÍA	MATERIAL	MALLA (MESH)
Forma / Vértices	Color / Textura	Objeto Visible en Escena

Definición: Un Mesh es el objeto 3D que se añade a la escena mediante `scene.add(mesh)`.

Sintaxis Básica: `const mesh = new THREE.Mesh(geometria, material);`

Anatomía de la Geometría 3D

¿De qué están hechos los objetos en el pipeline gráfico?



1 Vértices (Vertices)

Puntos en el espacio tridimensional con coordenadas (x, y, z).

2 Aristas (Edges)

Líneas vectoriales que conectan dos vértices.

3 Caras (Faces / Triángulos)

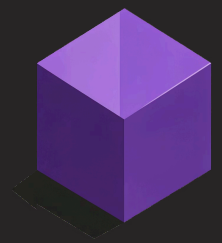
Superficies planas formadas por 3 vértices.

4 Normales (Normals)

Vectores perpendiculares a la superficie que determinan cómo rebota la luz.

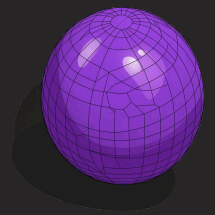
Geometrías Primitivas en Three.js (Parte 1)

Formas matemáticas integradas de uso frecuente



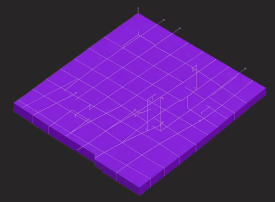
BoxGeometry

Cubos y paralelepípedos. `(width, height, depth)`



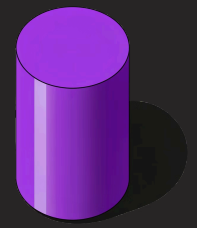
SphereGeometry

Esferas configurables. `(radius, widthSegments, heightSegments)`



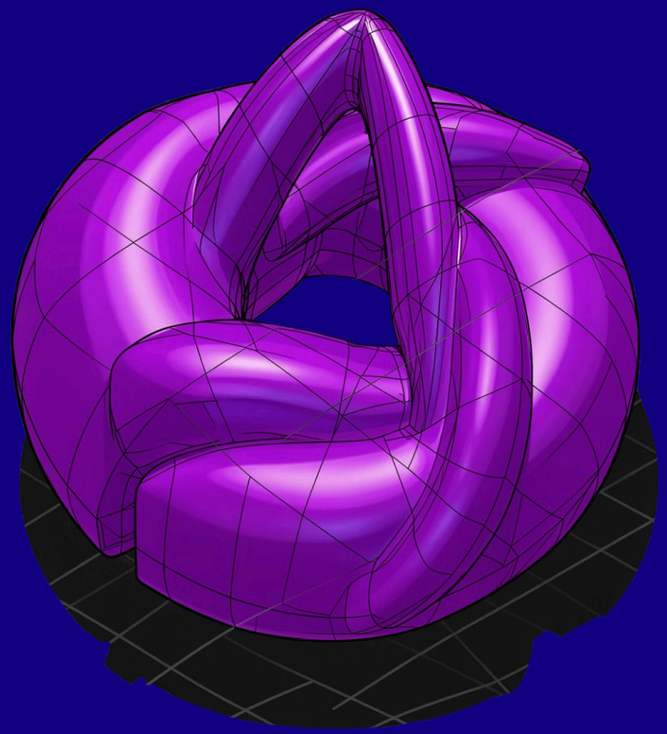
PlaneGeometry

Planos 2D para suelos o paredes. `(width, height)`



CylinderGeometry

Cilindros y conos. `(radiusTop, radiusBottom, height)`



Geometrías Primitivas en Three.js (Parte 2)

Formas avanzadas y complejas integradas

TorusGeometry

Anillos o rosquillas 3D.

`(radius, tube, radialSegments)`

TorusKnotGeometry

Nudos complejos ideales para pruebas de iluminación avanzada.

ConeGeometry

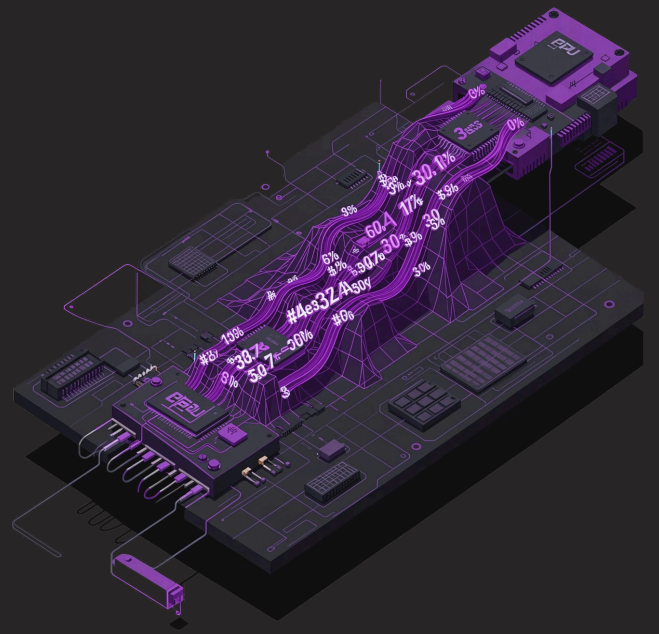
Conos ideales para vegetación o marcadores en escena.

Optimizaciones

A mayor número de segmentos, mayor suavidad visual pero mayor costo computacional.

Creación de Geometrías Personalizadas con BufferGeometry

Control total nivel bajo de vértices mediante arreglos tipados



BufferGeometry

La forma más eficiente de gestionar geometrías en memoria GPU.

Float32Array

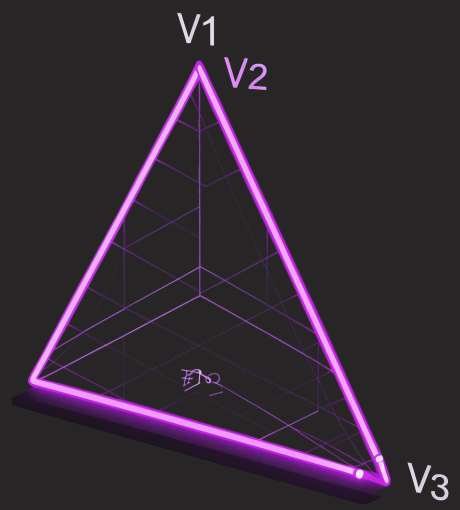
Arreglo optimizado de JavaScript para almacenar coordenadas de vértices.

Uso Práctico

Creación de terrenos procedurales, sistemas de partículas o polígonos irregulares.

Código: Creando un Triángulo Personalizado

Ejemplo de creación de geometría manual con BufferAttribute




```
const geometry = new THREE.BufferGeometry();

// 3 vértices x 3 coordenadas (x, y, z)
const vertices = new Float32Array([
  -1.0, -1.0, 0.0,  // Vértice 1
   1.0, -1.0, 0.0,  // Vértice 2
   0.0,  1.0, 0.0   // Vértice 3
]);

geometry.setAttribute(
  'position',
  new THREE.BufferAttribute(vertices, 3)
);
```

Introducción a los Materiales

Definiendo la reacción óptica de las superficies

Color Base

Propiedad `albedo` que define el tono fundamental de la superficie.

Opacidad y Transparencia

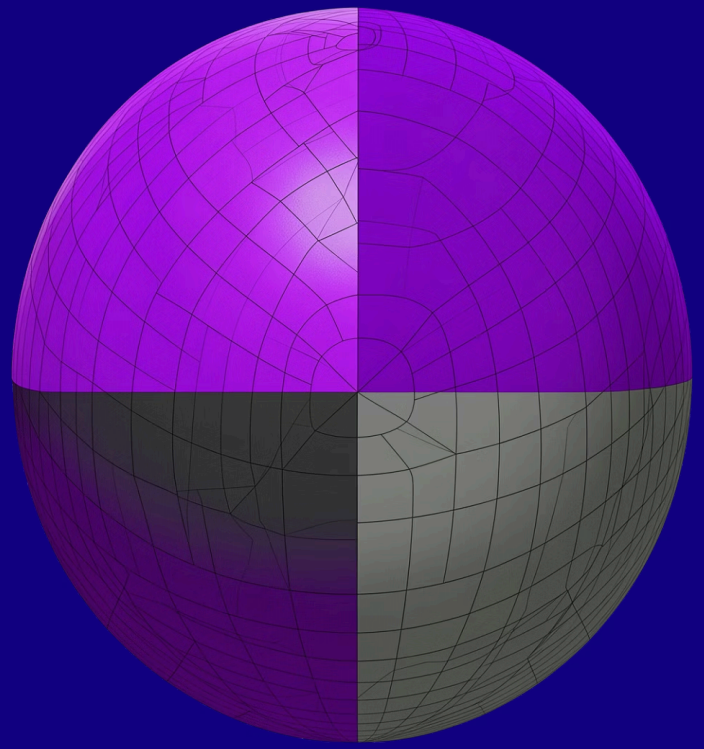
Propiedades `transparent: true` y `opacity: 0.5` para superficies semitransparentes.

Wireframe

Renderizado exclusivo del armazón de líneas vectoriales con `wireframe: true`.

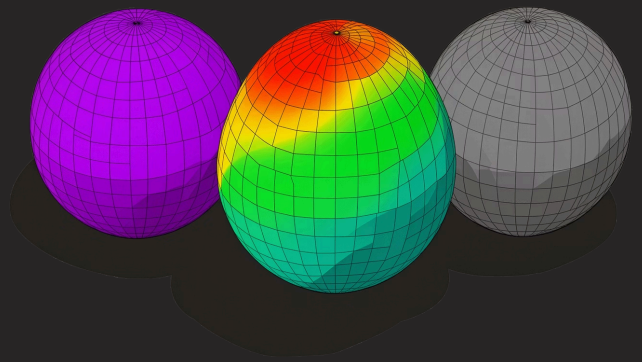
Material = Shader

Algoritmo de sombreado que determina cómo responde la superficie a la luz.

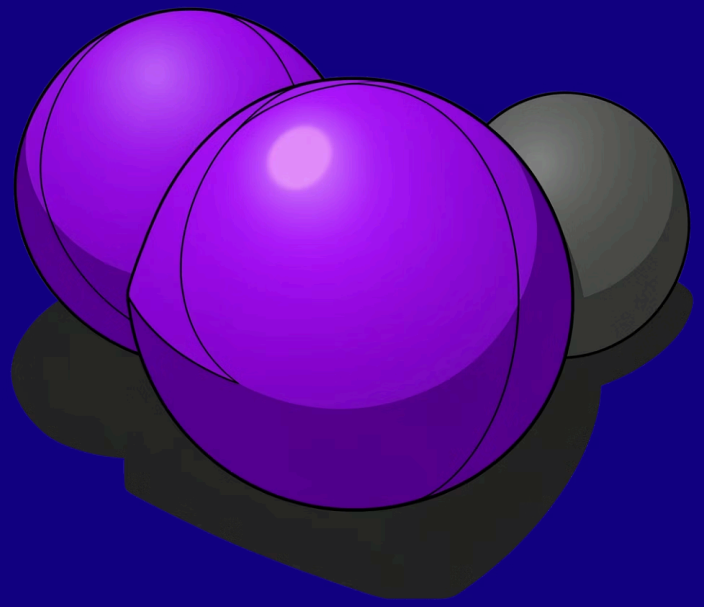


Tipos de Materiales: Básicos vs. Reactivos

Elegir el material correcto según el rendimiento y realismo



1	2	3
MeshBasicMaterial No requiere luces. Ideal para pruebas, pero luce plano y sin volumen.	**MeshNormalMaterial** Mapea los vectores normales a colores RGB. Excelente para depuración.	**MeshDepthMaterial** Dibuja la distancia del objeto respecto a la cámara (profundidad).



Materiales Avanzados con Sombras y Brillo

Agregando realismo clásico (Modelos Phong y Lambert)

MeshLambertMaterial

Sombreado mate sin reflejos especulares. Súper eficiente en rendimiento.

MeshPhongMaterial

Soporta reflejos brillantes y puntos de luz (`Shininess`).

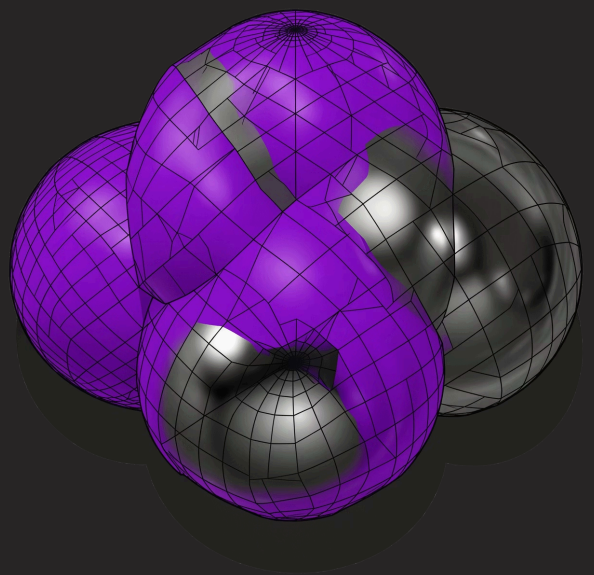
MeshToonMaterial

Sombreado estilo caricatura / Cel-shading con bordes definidos.

⚠ Requisito Obligatorio: Estos materiales requieren la presencia de al menos una luz (THREE.Light) en la escena para funcionar correctamente.

Materiales PBR (Physically Based Rendering)

El estándar de la industria moderna para gráficos 3D



MeshStandardMaterial

Material PBR estándar en Three.js basado en física real.

Metalicidad (metalness)

Controla qué tan metálica es la superficie. Rango: `0.0` a `1.0`.

Rugosidad (roughness)

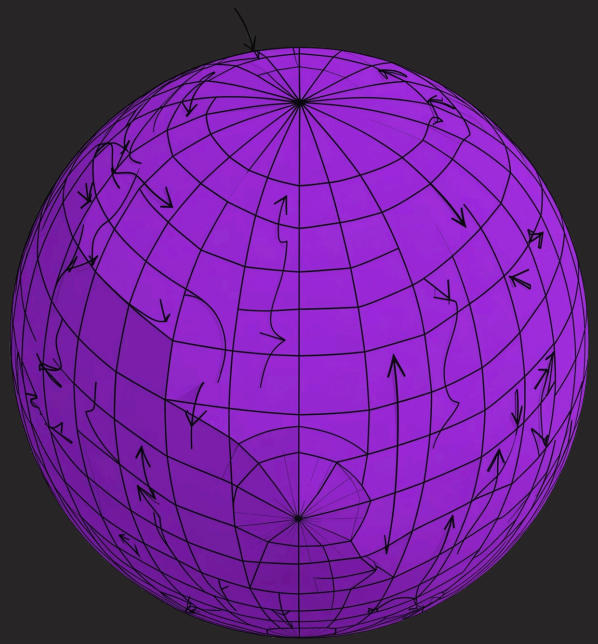
Controla el nivel de dispersión o pulido de la luz. Rango: `0.0` a `1.0`.

MeshPhysicalMaterial

Extensión del estándar con soporte para vidrio, barniz y difracción.

Carga y Aplicación de Texturas

Mapeo de imágenes sobre geometrías 3D

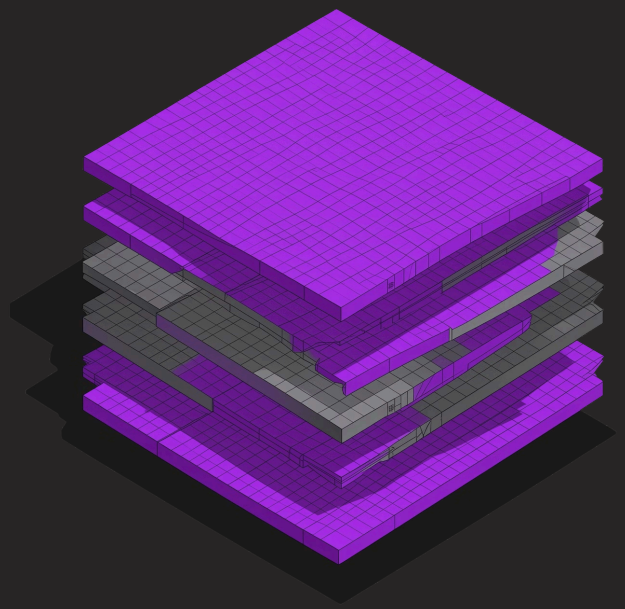


- **TextureLoader:** Clase encargada de descargar e interpretar imágenes (.jpg, .png).
- **Mapas de Color (map):** La imagen principal proyectada en la superficie.
- **UV Mapping:** Coordenadas 2D (u, v) que le indican al motor cómo envolver la imagen sobre la malla 3D.

```
const textureLoader = new THREE.TextureLoader();
const colorTexture = textureLoader.load(
  '/textures/wood_color.jpg'
);
const material = new THREE.MeshStandardMaterial({
  map: colorTexture
});
```

Mapeo de Texturas PBR Avanzado

Creando relieve y detalle microestructural sin añadir polígonos



Mapa de Normales (normalMap)

Simula relieve y rugosidad detallada afectando el rebote de la luz sin añadir geometría.

Mapa de Oclusión Ambiental (aoMap)

Agrega sombras oscuras en grietas y esquinas para mayor profundidad visual.

Mapa de Rugosidad (roughnessMap)

Controla qué partes de la textura son opacas o brillantes de forma granular.

Transformaciones 3D: Posición (Position)

Desplazamiento del objeto en el espacio tridimensional

Vector3

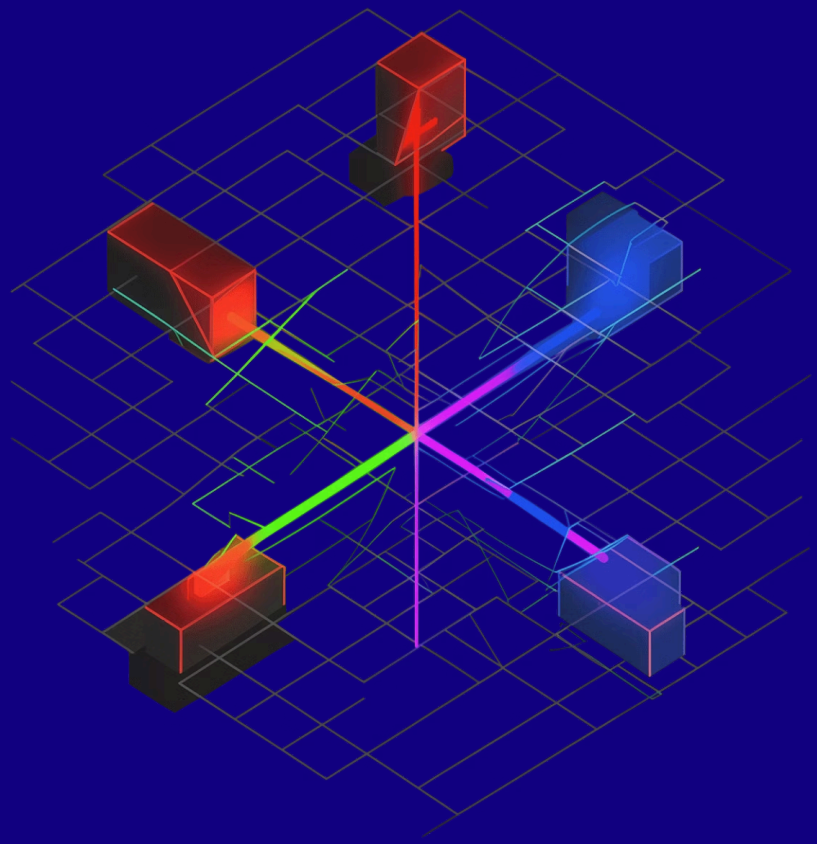
Propiedad `mesh.position` que contiene las coordenadas x, y, z del objeto en la escena.

Unidades

En Three.js, 1 unidad equivale arbitrariamente a 1 metro en el mundo virtual.

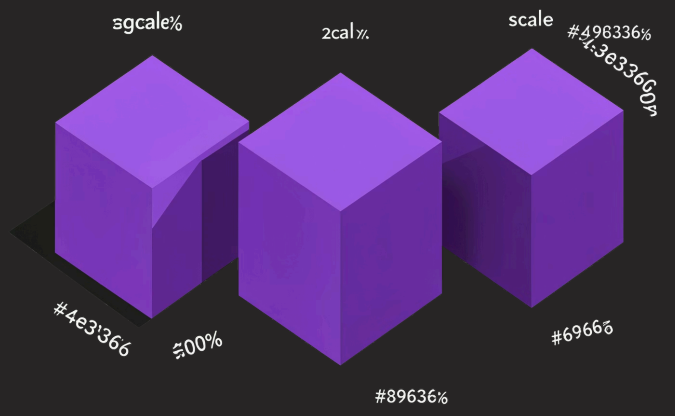
Sintaxis de Desplazamiento

```
mesh.position.x = 2;
mesh.position.set(2, 1, -5);
mesh.position.distanceTo(camera.position);
```



Transformaciones 3D: Escala (Scale)

Redimensionamiento relativo de la geometría



Vector3 de Escala

Propiedad `mesh.scale` con valores por defecto `(1, 1, 1)`.

Escala Uniforme

Cambiar todos los ejes por igual: `mesh.scale.set(2, 2, 2)`

Escala No Uniforme

Deformación del objeto alterando un solo eje: `mesh.scale.y = 0.5`

Transformaciones 3D: Rotación y Cuaterniones

Orientación angular del objeto en el espacio

1

Euler (mesh.rotation)

Rotación medida en radianes sobre ejes X, Y, Z.

2

Conversión Grados → Radianes

Radianes = Grados × (π/180).

Ej: `mesh.rotation.y = Math.PI * 0.25;` (45°)

3

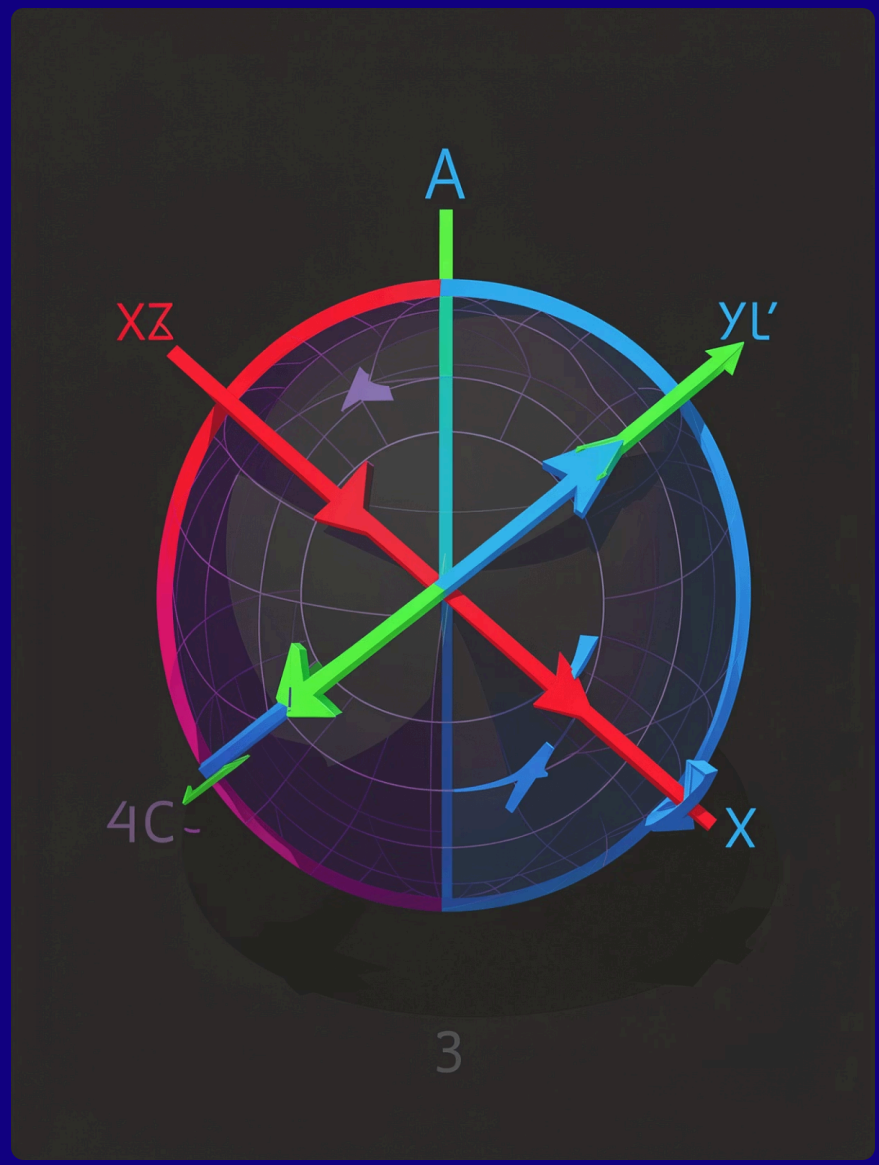
Gimbal Lock

Problema al rotar múltiples ejes en Euler (Bloqueo de Cardán).

4

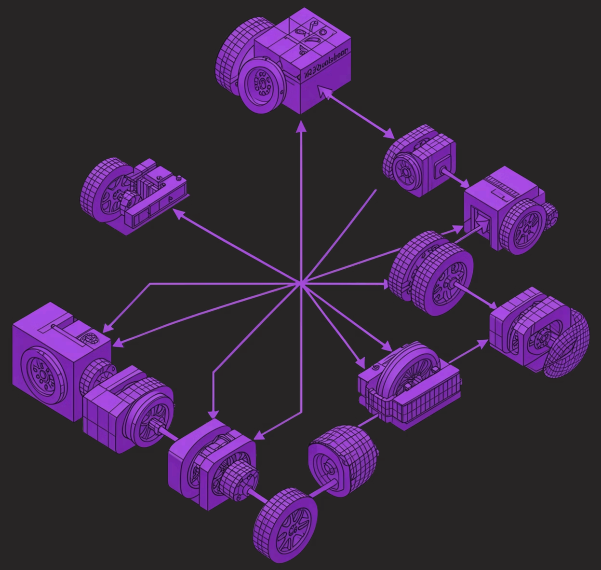
Cuaterniones (mesh.quaternion)

Representación matemática de 4 dimensiones para rotaciones complejas sin bloqueo.



Grupos de Objetos (THREE.Group)

Jerarquía y transformaciones compuestas



Utilidad: Permite agrupar múltiples Mesh para moverlos, rotarlos o escalarlos como una sola entidad.

Estructura Jerárquica: Las transformaciones del padre afectan automáticamente a todos los hijos.

```
const cocheGroup = new THREE.Group();
cocheGroup.add(chasisMesh);
cocheGroup.add(ruedaFrontalMesh);
scene.add(cocheGroup);

// Mueve todo el coche completo
cocheGroup.position.x = 10;
```

Laboratorio Práctico: Construcción de un Entorno 3D Interactivo

Paso a Paso Completo de Configuración y Desarrollo

Paso 0.1: Verificación de Entorno e Instalación de Node.js

Objetivo: Garantizar que el sistema reconozca el gestor de paquetes npm.

Verificación: Abrir PowerShell y ejecutar:

```
node -v
npm -v
```

- **Si npm no se reconoce: Descargar la versión LTS de Node.js desde nodejs.org.**
- **Asegurarse de marcar la casilla "Add to PATH" durante el asistente de instalación.**
- **Obligatorio: Cerrar la consola y volver a abrirla para aplicar los cambios.**

Paso 0.2: Solución al Error de Políticas de PowerShell (PSSecurityException)

Problema: Ocurre al ejecutar npm si la ejecución de scripts está bloqueada por Windows.

Solución: Abrir PowerShell como Administrador y ejecutar:

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Confirmación: Presionar la tecla `S` (Sí) y luego Enter. Confirmar ejecutando `npm -v`.

Paso 1 y 2: Creación del Proyecto e Instalación de Dependencias

Paso 1: Creación del Proyecto con Vite

Navegar a la carpeta contenedora en PowerShell e inicializar la plantilla Vanilla JS:

```
cd C:\Ruta\De\Tu\Proyecto
npm create vite@latest lab-semana2 -- --template vanilla
```

Si deseas crear el proyecto directamente dentro de la carpeta actual sin subcarpetas, usa el punto (.):

```
npm create vite@latest . -- --template vanilla
```

Paso 2: Instalación de Dependencias 3D (Three.js + lil-gui)

Acceder a la carpeta del proyecto e instalar las librerías necesarias:

```
cd lab-semana2
npm install three lil-gui
```

Verificación: Se creará la carpeta `node_modules` y el archivo `package.json` registrará las dependencias `three` y `lil-gui`.

Paso 3: Configuración de Estilos Globales (src/style.css)

Abrir la carpeta en VS Code y reemplazar todo el contenido de `src/style.css` por:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  overflow: hidden;
  background-color: #1e1e2e;
}
```

Paso 4 (Parte 1): Escena, Cámara, Renderizador e Iluminación PBR

Desarrollo del Escenario 3D PBR Completo — `src/main.js`

Borrar todo el contenido de `src/main.js` y pegar el siguiente código por partes:

```
import * as THREE from 'three';
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js';
import GUI from 'lil-gui';

// 1. ESCENA, CÁMARA Y RENDERIZADOR
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x1e1e2e);

const camera = new THREE.PerspectiveCamera(75, window.innerWidth / window.innerHeight, 0.1, 100);
camera.position.set(0, 4, 8);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
document.body.appendChild(renderer.domElement);

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;

// 2. ILUMINACIÓN (PBR)
const ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
const directionalLight = new THREE.DirectionalLight(0xffffff, 1.2);
directionalLight.position.set(5, 8, 5);
scene.add(ambientLight, directionalLight);

// 3. SUELO CON CUADRÍCULA
const gridHelper = new THREE.GridHelper(12, 12, 0x888888, 0x444444);
scene.add(gridHelper);
```

Paso 4 (Parte 2): Meshes PBR y Controles lil-gui

Continuación de `src/main.js` — Geometrías, Materiales y Panel de Control

```
// 4. MESHES Y MATERIALES PBR
// Esfera (Metal Pulido)
const sphereGeo = new THREE.SphereGeometry(0.8, 32, 32);
const sphereMat = new THREE.MeshStandardMaterial({ color: 0xe74c3c, metalness: 0.9, roughness: 0.1 });
const sphere = new THREE.Mesh(sphereGeo, sphereMat);
sphere.position.set(-2.5, 1.2, 0);

// Nudo Toroidal (Plástico Brillante)
const knotGeo = new THREE.TorusKnotGeometry(0.6, 0.2, 100, 16);
const knotMat = new THREE.MeshStandardMaterial({ color: 0x3498db, metalness: 0.1, roughness: 0.2 });
const knot = new THREE.Mesh(knotGeo, knotMat);
knot.position.set(0, 1.2, 0);

// Cilindro (Mate / Dieléctrico)
const cylGeo = new THREE.CylinderGeometry(0.7, 0.7, 1.5, 32);
const cylMat = new THREE.MeshStandardMaterial({ color: 0x2ecc71, metalness: 0.0, roughness: 0.8 });
const cylinder = new THREE.Mesh(cylGeo, cylMat);
cylinder.position.set(2.5, 1.2, 0);

scene.add(sphere, knot, cylinder);

// 5. CONTROLES EN VIVO CON LIL-GUI
const gui = new GUI();

const f1 = gui.addFolder('1. Esfera (Metal)');
f1.add(sphereMat, 'metalness', 0, 1, 0.05).name('Metalicidad');
f1.add(sphereMat, 'roughness', 0, 1, 0.05).name('Rugosidad');
f1.add(sphere.position, 'y', 0.8, 4, 0.1).name('Posición Y');

const f2 = gui.addFolder('2. Nudo (Plástico)');
f2.add(knotMat, 'metalness', 0, 1, 0.05).name('Metalicidad');
f2.add(knotMat, 'roughness', 0, 1, 0.05).name('Rugosidad');
f2.add(knot.position, 'y', 0.8, 4, 0.1).name('Posición Y');

const f3 = gui.addFolder('3. Cilindro (Mate)');
f3.add(cylMat, 'metalness', 0, 1, 0.05).name('Metalicidad');
f3.add(cylMat, 'roughness', 0, 1, 0.05).name('Rugosidad');
f3.add(cylinder.position, 'y', 0.8, 4, 0.1).name('Posición Y');
```

Paso 4 (Parte 3) y Paso 5: Bucle de Animación y Resultado Final

Continuación de `src/main.js` — Animación 60 FPS, Resize y Ejecución

Paso 4 (Parte 3): Bucle de Animación y Resize

```
// 6. BUCLE DE ANIMACIÓN (60 FPS)
function animate() {
  requestAnimationFrame(animate);
  sphere.rotation.x += 0.01;
  knot.rotation.y += 0.01;
  cylinder.rotation.z += 0.01;
  controls.update();
  renderer.render(scene, camera);
}
animate();

window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
```

Paso 5: Ejecución y Resultado Final

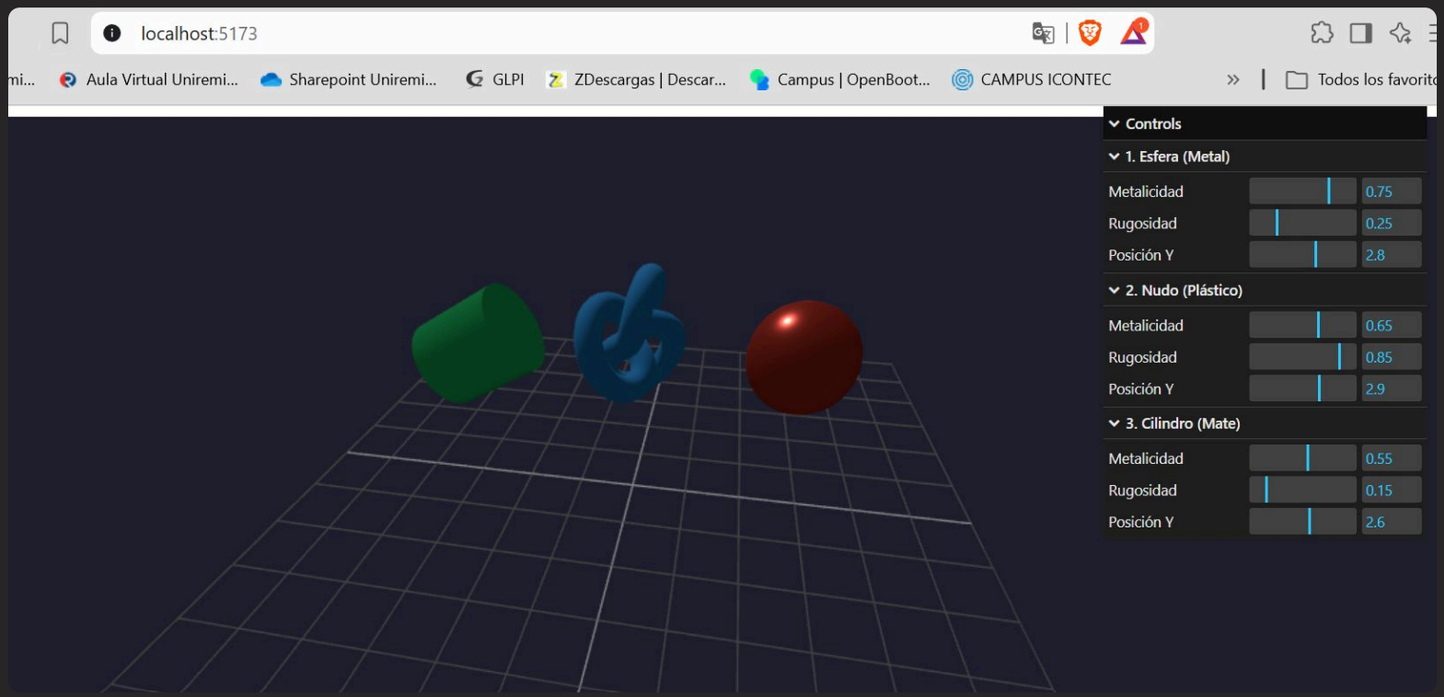
```
npm run dev
```

Efecto Final Observado:

- Acceso: Abrir el navegador e ingresar a `http://localhost:5173/`
- Tres objetos 3D flotando sobre la rejilla, cada uno rotando dinámicamente en ejes diferentes a 60 FPS.
- Variación clara de respuestas a la luz (reflejo especular en la esfera metálica vs. aspecto opaco en el cilindro mate).
- Panel flotante lil-gui en la esquina superior derecha que permite manipular la altura y propiedades físicas PBR de cada objeto en tiempo real.

Paso 4 : Resultado Final

Al ejecutar el localhost:5173 este resultado se deberia visualizar.



Efecto Final Observado:

- Acceso: Abrir el navegador e ingresar a `http://localhost:5173/`
- Tres objetos 3D flotando sobre la rejilla, cada uno rotando dinámicamente en ejes diferentes a 60 FPS.
- Variación clara de respuestas a la luz (reflejo especular en la esfera metálica vs. aspecto opaco en el cilindro mate).
- Panel flotante lil-gui en la esquina superior derecha que permite manipular la altura y propiedades físicas PBR de cada objeto en tiempo real.

Reto Práctico para la Clase (45 minutos)

Construcción de una Composición Multimedia Interactiva

MISIÓN

01

Escenario Base

Crear un escenario con un suelo (`PlaneGeometry`) y 3 objetos 3D distintos en el aire.

02

Materiales Diferenciados

Aplicar un `MeshStandardMaterial` diferente a cada objeto: un metal pulido, un plástico brillante y un objeto mate.

03

Bucle de Animación

Crear un bucle con `requestAnimationFrame` que rote los objetos en ejes diferentes a 60 FPS.

04

Paneles lil-gui

Agregar paneles para modificar el color y rugosidad de los 3 objetos independientemente.

Entregable: Demostración en pantalla del entorno 3D funcionando con controles dinámicos.

Conclusiones y Próxima Clase

Puntos clave aprendidos y mirada hacia la Semana 3

Mallas = Geometría + Material

La geometría da la estructura de polígonos y el material define la respuesta física a la luz.

Transformaciones Fundamentales

`position`, `rotation` y `scale` forman la matriz de transformación fundamental de todo objeto 3D.

Siguiente Paso — Semana 3

Medios, Animación y Audio: Bucle a 60 FPS, mapas de video como textura y configuración de audio espacial 3D.